

A template/ approach to Reproducible Generative Research

Daniel Ari Friedman

Active Inference Institute

daniel@activeinference.institute

ORCID: 0000-0001-6232-9096

DOI: 10.5281/zenodo.20419007

2026-06-27

BEGINNING OF TRANSMISSION

State: published

Pairing: complete (DOI, GitHub, SHA-256, Zenodo URL)

Release metadata

Field	Value
Title	A template/ approach to Reproducible Generative Research
Version	1.0.9
Concept DOI	10.5281/zenodo.20419007
Version DOI	10.5281/zenodo.20976048
GitHub	https://github.com/docxology/template_template/releases/tag/v1.0.9
Zenodo	https://zenodo.org/records/20419007
SHA-256	535bd80943d0ae9f...
SHA-512	pending

How to verify

- Scan **Integrity** QR and compare the embedded SHA-256 prefix to the table above.
- Scan **Zenodo** / **GitHub** QR codes and confirm they resolve to this release pairing.
- Full hashes and structured fields: ../data/transmission_manifest.json.

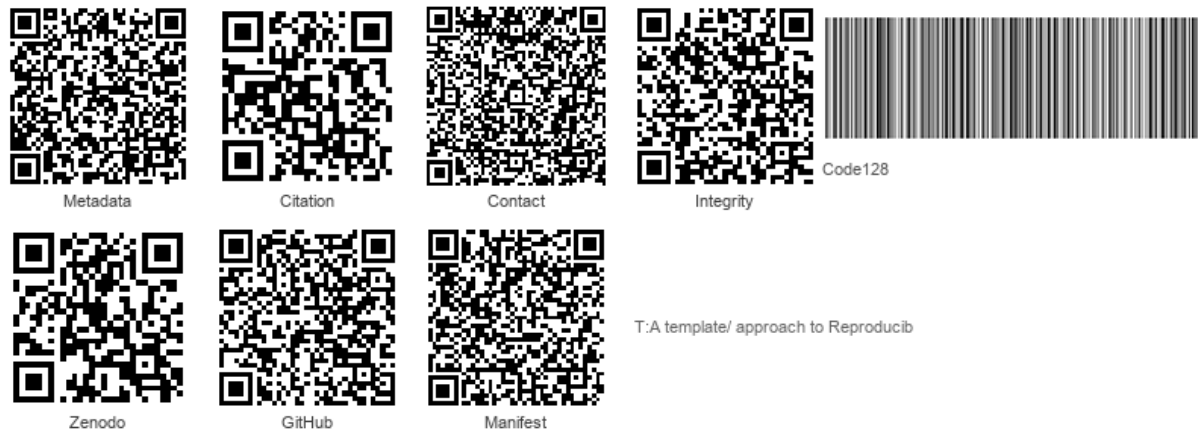


Figure 1: Integrity QR strip

Structured manifest: ../data/transmission_manifest.json

Stego: off | overlays text | barcodes on | XMP on | manifest on → ../secure_run.sh

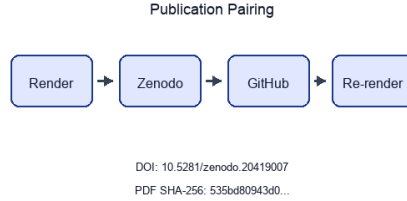


Figure 2: Publication pairing flow

Contents

1	Methods	4
1.1	The Two-Layer Architecture	4
1.2	The Standalone Project Paradigm	4
1.3	The Thin Orchestrator Pattern	5
1.4	DAG Pipeline Declared by <code>pipeline.yaml</code>	6
1.4.1	Stage Highlights	6
1.4.2	Interactive Orchestration	6
1.5	Documentation Duality and AI Collaboration	8
1.6	Agentic Skill Architecture	8
1.6.1	The Three-Tier Skill Protocol	8
1.6.2	Module Skill Coverage	9
1.6.3	MCP Server Mapping	9
1.7	FAIR Alignment and Research Infrastructure as Code	10
1.7.1	Principle-by-Principle Alignment	10
1.7.2	Infrastructure as Code for Research	10
1.8	Quality Assurance	11
1.8.1	Zero-Mock Testing Policy	11
1.8.2	Coverage Thresholds	11
1.8.3	Test Suite Composition	11
1.8.4	Visualization Standards	11
2	Discussion	13
2.1	The Zero-Mock Tradeoff	13
2.1.1	When Mocks Are Not the Problem	13
2.1.2	Practical Implementation	13
2.2	Comparison to Existing Tools	14
2.2.1	FAIR4RS Evolution (2024–2026)	14
2.3	The AI Collaboration Model	15
2.4	The Learning Curve	15
2.5	Limitations	15
2.6	Future Directions	17
2.7	Conclusion	17
3	Infrastructure Module Reference	19
3.1	Alphabetical summaries	20
3.1.1	<code>infrastructure.autoresearch</code> (10 files)	20
3.1.2	<code>infrastructure.benchmark</code> (4 files)	20
3.1.3	<code>infrastructure/config</code> (non-package subdirectory)	20
3.1.4	<code>infrastructure.core</code> (120 files)	20
3.1.5	<code>infrastructure.doctor</code> (14 files)	21
3.1.6	<code>infrastructure.docker</code> (0 files)	21

3.1.7	infrastructure.documentation (14 files)	21
3.1.8	infrastructure.fonds (6 files)	21
3.1.9	infrastructure.llm (55 files)	21
3.1.10	infrastructure.methods (5 files)	21
3.1.11	infrastructure.orchestration (12 files)	21
3.1.12	infrastructure.project (41 files)	21
3.1.13	infrastructure.prose (9 files)	21
3.1.14	infrastructure.provenance (7 files)	21
3.1.15	infrastructure.publishing (81 files)	21
3.1.16	infrastructure.reference (16 files)	21
3.1.17	infrastructure.rendering (65 files)	22
3.1.18	infrastructure.reporting (57 files)	22
3.1.19	infrastructure.research (3 files)	22
3.1.20	infrastructure.rules (6 files)	22
3.1.21	infrastructure.scientific (4 files)	22
3.1.22	infrastructure.search (62 files)	22
3.1.23	infrastructure.sia (10 files)	22
3.1.24	infrastructure.skills (8 files)	22
3.1.25	infrastructure.steganography (13 files)	22
3.1.26	infrastructure.tools (6 files)	22
3.1.27	infrastructure.validation (80 files)	22
3.1.28	infrastructure/logrotate.d (0 files)	23
4	Appendices	24
4.1	Appendix: Pipeline Stage Reference	24
4.2	Appendix: Configuration Reference	26
4.3	Appendix: Repository Directory Structure	28
4.4	Appendix: Documentation Inventory	29
4.5	Appendix: Comparative Tool Matrix	30

1 Methods

The `template/` architecture is deliberately bifurcated into a globally shared `infrastructure/` layer and project-specific `projects/` silos. This section describes the four core design patterns, the YAML-declared pipeline (16 stages; default full 10) pipeline that operationalizes them, and the AI collaboration model that distinguishes this system from conventional research templates.

1.1 The Two-Layer Architecture

The repository is organized into two strictly separated layers:

Infrastructure Layer (`infrastructure/`): 28 infrastructure subdirectories—25 of them independently-importable Python packages—comprising about 708 modules and providing reusable services. Each importable package has its own `__init__.py`, `AGENTS.md`, and `README.md`, and exports a well-defined public API (the remaining subdirectories, e.g. `config/`, hold shared configuration). The infrastructure layer knows nothing about any specific project—it provides generic capabilities (logging, rendering, validation, steganography) that any project may consume.

Project Layer (`projects/`): Self-contained research workspaces. Each project directory contains:

Directory	Purpose
<code>manuscript/</code>	Markdown chapters and <code>config.yaml</code>
<code>scripts/</code>	Thin orchestrator scripts (Stage 02)
<code>src/</code>	Project-specific Python modules
<code>tests/</code>	Project-specific test suite
<code>data/</code>	Input datasets and generated data
<code>output/</code>	Pipeline artifacts: PDF, figures, reports, logs
<code>docs/</code>	Project-specific architecture documentation

The two layers communicate exclusively through Python imports and filesystem paths. No project modifies infrastructure code; no infrastructure module references a specific project by name (except via runtime project discovery).

1.2 The Standalone Project Paradigm

Projects are designed to be completely self-contained. Adding a new project requires no changes to the infrastructure layer, no modifications to `pyproject.toml`, and no updates to the pipeline orchestrator. A project is automatically discovered if and only if it satisfies two conditions:

1. It exists as a subdirectory of `projects/`.
2. It contains the file `manuscript/config.yaml`.

This paradigm enables horizontal scaling: N researchers can maintain N independent projects within a single repository, sharing infrastructure without coupling. Each project declares its own testing tolerances, manuscript metadata, LLM review preferences, and rendering configuration in its `config.yaml`. The system currently hosts its public canonical exemplars under `projects/templates/` (`templates/template_active_inference`, `templates/template_advanced_literature_review`, `templates/template_autopoiesis`, `templates/template_autoresearch_project`, `templates/template_autoscientists`, `templates/template_code_project`, `templates/template_data_descriptor`, `templates/template_eda_notebook`, `templates/template_formal`, `templates/template_gold_refinement`, `templates/template_literature_meta_analysis`, `templates/template_madlib`, `templates/template_methods_paper`, `templates/template_newspaper`, `templates/template_pitch_deck`, `templates/template_pools_rules_tools`, `templates/template_prose_project`, `templates/template_redacted_report`, `templates/template_registered_report`, `templates/template_search_project`, `templates/template_sia`, `templates/template_storybook`, `templates/template_template`, `templates/template_textbook`), including this meta-manuscript at `projects/templates/template_template/`.

1.3 The Thin Orchestrator Pattern

All scripts in `scripts/` (both infrastructure-level and project-level) follow the Thin Orchestrator pattern [Gamma et al., 1995]:

- **No domain logic:** Scripts contain zero algorithmic implementation. They import functions from `src/` modules and wire them to infrastructure services.
- **Configuration-driven:** Behavior is parameterized by `config.yaml`, not by hardcoded values.
- **Stateless:** Scripts read inputs, call functions, write outputs. They maintain no persistent state between invocations.
- **Logged:** Every significant action is logged via `infrastructure.core.logging.utils.get_logger`.

This pattern ensures that all testable logic lives in `src/` where it is subject to the Zero-Mock testing policy, while scripts remain thin enough to be audited by visual inspection. The separation draws on the Mediator pattern from Gamma et al. [Gamma et al., 1995], where scripts mediate between infrastructure services and project-specific code without implementing any logic of their own.

To make this concrete, the following contrasts the anti-pattern with the correct pattern:

```
# ANTI-PATTERN: domain logic embedded in script
def calculate_average(data):                # ← never put computation here
    return sum(data) / len(data)

result = calculate_average([1, 2, 3])

# CORRECT PATTERN: script imports from src/ and only wires
from projects.my_project.src.statistics import calculate_average

result = calculate_average([1, 2, 3]) # ← scripts wire, never compute
```

The critical property is that `calculate_average` in the correct pattern lives in a testable `src/` module, is covered by the Zero-Mock test suite, and can be independently imported, tested, and reused—whereas the anti-pattern buries logic in a script that is invisible to coverage tools.

1.4 DAG Pipeline Declared by `pipeline.yaml`

Single-project pipelines read `infrastructure/core/pipeline/pipeline.yaml`. `scripts/runner/execute_pipeline.py` expands the declarative DAG, applies tag filters (`--core-only` skips llm stages), checkpoints between nodes, then dispatches the declared `scripts/pipeline/stage_NN*.py` scripts or builtin methods (`_run_clean_outputs`).

The **default YAML graph contains ten named core stages** (plus telemetry configuration metadata):

1. **Clean Output Directories** — wipes prior projects/`<name>/output/` + delivered output/`<name>/` paths so stale PDFs cannot satisfy validation.
2. **Environment Setup** (`scripts/pipeline/stage_00_setup.py`) — Python/uv probing, toolchain discovery, scaffolding directories, PYTHONPATH wiring.
3. **Infrastructure Tests** (`scripts/pipeline/stage_01_test.py --infra-only`) — tests/ suite with the configured $\geq 60\%$ source-coverage floor.
4. **Project Tests** (`scripts/pipeline/stage_01_test.py --project-only`) — isolated per-project suites with each project's declared floor ($\geq 90\%$ for this exemplar).
5. **Project Analysis** (`scripts/pipeline/stage_02_analysis.py`) — lexicographically ordered projects/`<name>/scripts/*.py`, each a thin orchestrator (`src/` does real work).
6. **PDF Rendering** (`scripts/pipeline/stage_03_render.py`) — Pandoc $\rightarrow$ XeLaTeX loop, bibliography assembly, injected variables from Stage 02 artefacts.
7. **Output Validation** (`scripts/pipeline/stage_04_validate.py`) — PDF structure, manifests, Markdown hygiene.
8. **LLM Scientific Review** (`scripts/pipeline/stage_06_llm_review.py --reviews-only; tags: llm`) — executive + quality critiques via local Ollama; `allow_skip: true`.
9. **LLM Translations** (`scripts/pipeline/stage_06_llm_review.py --translations-only; tags: llm`, same dependency edges) — multilingual abstract expansion.
10. **Copy Outputs** (`scripts/pipeline/stage_05_copy.py`) — reproducible snapshots into canonical output/`<project>/`.

Two LLM nodes intentionally share one script module with orthogonal CLI switches; both depend only on validation so they can parallelize logically while remaining optional.

Executive reporting (`scripts/pipeline/stage_07_executive_report.py`) is **not** a YAML node inside the single-project executor. `--all-projects / execute_multi_project.py` invokes it once after iterating projects, consolidating cross-project KPIs dashboards.

Topological order therefore differs slightly from declaration order (e.g., copy executes after validation even though stage `stage_05_copy.py` precedes `stage_06_llm_review.py` lexically).

1.4.1 Stage Highlights

Infrastructure vs project tests. Splitting pytest invocations isolates flaky infra regressions (`MAX_TEST_FAILURES` knobs) from zero-tolerance gates on domain code (`max_project_test_failures` default 0 declared in YAML front-matter/testing blocks).

Stage 02 illustration. The analysis stage is deliberately concrete rather than a hypothetical diagram factory: each canonical project ships real behaviour at this node. `template_autoresearch_project` runs readiness validation; `template_search_project` merges remote literature JSON, generates scripted figures (`y_generate_search_figures.py`), and writes manifests; `template_code_project` emits optimization plots; and `template_prose_project` triggers structural validation scaffolding. The pipeline shape is identical across all four—only the Stage 02 payload differs—which is exactly what lets one orchestrator serve heterogeneous research domains.

1.4.2 Interactive Orchestration

1.4.2.1 `run.sh` Thin wrapper invoking `python -m infrastructure.orchestration`. Offers:

- per-project staged execution,
- chained digits (234 shorthand),
- multi-project grid (`a-d` presets),
- graceful quit / resume parity with `scripts/README.md`.

Selecting **d alone** after a passing multi-project run exits immediately once summaries print—avoiding repetitive menu redraw.

1.4.2.2 secure_run.sh Executes Python **secure** path: standard pipeline artefact reproduction **then** invokes **run_secure_pipeline** for steganographic PDF hardening (**infrastructure.steganography**). Original PDFs stay immutable; hardened companions carry QR overlays plus hash manifests sidecars.

1.5 Documentation Duality and AI Collaboration

Every directory at every level of the repository hierarchy contains two documentation files:

- **README.md**: Human-readable overview, quick-start instructions, and directory structure.
- **AGENTS.md**: Machine-readable technical specification optimized for AI coding assistants. Contains API tables, dependency graphs, implementation patterns, and architectural constraints.

This Documentation Duality standard serves two purposes. First, it ensures that both human researchers and AI agents can navigate the codebase efficiently—**AGENTS.md** files provide the structured context that language models need to make informed code modifications without hallucinating API signatures or violating architectural invariants. Second, it creates a self-documenting feedback loop: as AI agents modify the codebase, they update the corresponding **AGENTS.md** files, keeping documentation synchronized with implementation. Lau and Guo’s survey of 90 AI coding assistant systems [Lau and Guo, 2025] identifies contextual code understanding as a primary bottleneck; the Documentation Duality standard addresses this by providing pre-structured context at every directory level.

The template additionally includes **CLAUDE.md** at the repository root, providing system-level instructions for AI coding assistants—architectural principles, testing requirements, and naming conventions that apply globally. This creates a three-tier documentation architecture: per-directory **AGENTS.md** for local context, root **README.md** and **CLAUDE.md** for global constraints, and **README.md** for human navigation.

1.6 Agentic Skill Architecture

The **Documentation Duality** standard addresses human and AI navigation at the directory level. A complementary layer operates at the *module* level: every infrastructure subpackage carries two additional machine-readable files that transform it from a passive code library into an active, protocol-aligned skill endpoint.

1.6.1 The Three-Tier Skill Protocol

`template/` implements a progression of agent-facing documentation, escalating in specificity from global constraints to module-level API contracts:

Tier	File	Scope	Purpose
1 — System	README.md	Repository root	Global architectural principles, Zero-Mock policy, naming conventions
2 — Structure	AGENTS.md	Every directory	Local file inventories, API surfaces, integration patterns, architectural constraints
3 — Skill	SKILL.md	Every infrastructure module	Machine-parseable skill descriptor: module name, description, key imports, usage pattern

Tier 1 and Tier 2 have direct analogues in the prompt-engineering literature: system prompts and retrieval-augmented context [Lau and Guo, 2025]. Tier 3 is novel. The **SKILL.md** files follow a YAML frontmatter + Markdown instruction format precisely aligned with the tool-descriptor schemas of the Model Context Protocol [Anthropic, 2024]. The following is the actual frontmatter from `infrastructure/rendering/SKILL.md`:

name: `infrastructure-rendering`

description: Skill for the rendering infrastructure module providing multi-format output generation inc.

An MCP client reading this block immediately knows the module name, its natural-language activation condition (“use for”), and which Python symbols to import. No source-code inspection is required. This is the practical implementation of Toolformer-style self-documented tools [Schick et al., 2023]—rather than a

language model learning tool APIs from training data, the APIs are encoded directly in version-controlled, co-located skill files that evolve with the codebase.

1.6.2 Module Skill Coverage

Each infrastructure subdirectory surfaced by `discover_infrastructure_modules()` carries paired `README.md` + `AGENTS.md`; agent-facing `SKILL.md` manifests exist wherever teams enable Cursor / PAI manifests (regenerated via `python -m infrastructure.skills`). Root-level `PAI.md` summarizes cross-package obligations.

Promotion policy: new Layer-1 directories must ship human + machine-readable docs (`README.md`, `AGENTS.md`) immediately; Tier-3 `SKILL` assets follow once APIs stabilize.

1.6.3 MCP Server Mapping

The mapping from `SKILL.md` descriptors to MCP server endpoints is intentional but not yet automated; it represents the principal next integration step. In the MCP architecture [Anthropic, 2024], a server exposes three primitive types: **Tools** (executable functions), **Resources** (data the model can read), and **Prompts** (structured query templates). Each `infrastructure` module maps naturally onto this taxonomy:

- `infrastructure.llm` → MCP **Tool** (`query`, `apply_template`) + MCP **Prompt** (research prompt templates)
- `infrastructure.rendering` → MCP **Tool** (`render_pdf`, `render_html`) + MCP **Resource** (rendered PDFs as URI-addressable resources)
- `infrastructure.validation` → MCP **Tool** (`validate_pdf_rendering`, `validate_markdown`)
- `infrastructure.publishing` → MCP **Tool** (`publish_to_zenodo`, `generate_citation_bibtex`) + MCP **Resource** (DOI registry)
- `infrastructure.steganography` → MCP **Tool** (`SteganographyProcessor.process`) + MCP **Resource** (hash manifests)
- `infrastructure.search` · `infrastructure.reference` → MCP **Tool** wrappers over literature retrieval + BibTeX handling + MCP **Resource** exports for corpus JSON / `.bib`

An agent orchestrating a full research pipeline could, in principle, compose these MCP tools to reproduce the declarative DAG programmatically—discovering capabilities via `SKILL.md` frontmatter, executing them via MCP protocol calls, and consuming their outputs as **Resources**. The `SKILL.md` files parallel Voyager’s skill library [Wang et al., 2023]—Voyager’s agent accumulates a growing library of executable Minecraft skills represented as JavaScript functions; `template/`’s agent accumulates a curated library of research pipeline skills represented as YAML-frontmattered `SKILL.md` files. In both cases, the skill representation is machine-readable, version-controlled, and self-describing. Wang et al.’s LLM agent survey [Wang et al., 2024a] identifies tool use, planning, and memory as the three fundamental capabilities of autonomous agents; Yao et al.’s ReAct framework [Yao et al., 2023] demonstrates that interleaving reasoning traces with tool actions dramatically improves agent reliability in interactive settings. The `template/` skill architecture maps cleanly onto these three capabilities: the `SKILL.md` descriptors supply the tool-use layer, the declarative DAG of `16 pipeline.yaml` stages (a default full run executes 10) supplies the planning scaffold, and the per-criterion checkpoint system supplies the memory layer.

1.7 FAIR Alignment and Research Infrastructure as Code

The template’s design aligns with both the original FAIR principles [Wilkinson et al., 2016] and the FAIR for Research Software (FAIR4RS) principles [Barker et al., 2022] at the repository level. FAIR4RS recognizes that software has requirements distinct from data—executability, composability, and dependency management—and the template addresses each.

1.7.1 Principle-by-Principle Alignment

Findability. Outputs are *Findable* through standardized directory structures, manifest files, and machine-readable metadata embedded in PDFs. Every project’s `config.yaml` provides structured metadata (title, authors, DOIs, keywords) in a format parseable by both Pandoc and external indexing services. The `metrics.json` output provides a machine-readable inventory of all generated artifacts, their locations, and their provenance hashes.

Accessibility. Outputs are *Accessible* via open-source distribution on GitHub, with metadata embedded in the artifact itself rather than in a separate registry. The steganographic layer embeds provenance information directly in the PDF—including SHA-256 content hashes, build timestamps, and QR-encoded metadata—ensuring accessibility even when the PDF circulates outside the repository.

Interoperability. *Interoperability* is achieved through standard formats (PDF, JSON, BibTeX, YAML) and well-defined module APIs that enable cross-project composition. The Pandoc rendering pipeline accepts any Markdown-with-LaTeX input conforming to the template’s section numbering conventions, allowing seamless migration of manuscripts from other Pandoc-based workflows.

Reusability. *Reusability* is ensured by the Standalone Project Paradigm—any project can be extracted and reused independently—and by the Documentation Duality standard, which satisfies FAIRsoft’s inspectability and documentation quality indicators [Garijo et al., 2024]. The pipeline’s automated testing and coverage enforcement directly operationalize the FAIR4RS executability requirement: software that cannot pass its own test suite cannot produce publishable output.

1.7.2 Infrastructure as Code for Research

At a higher level of abstraction, `template/` applies the DevOps principle of *Infrastructure as Code* (IaC) to the research lifecycle. In production software engineering, IaC means that server configuration is version-controlled, automatically provisioned, and independently reproducible [Wilson et al., 2017]. `template/` extends this principle to the research manuscript: the document is not authored in a word processor and emailed to collaborators, but *built* from version-controlled Markdown sources, *tested* against formal coverage thresholds, and *deployed* to a provenance-embedded PDF.

Every component of the research pipeline—the test suite, the analysis scripts, the rendering configuration, and the steganographic watermark—is specified in code, committed to git, and reproducible from a clean checkout. This deterministic build property means that any researcher can clone the repository, run `./run.sh --pipeline`, and produce a byte-for-byte identical manuscript (modulo timestamps in the steganographic metadata).

Software Heritage [Di Cosmo et al., 2020] provides persistent SWHIDs (Software Hash Identifiers) for source code snapshots, enabling stable citation of any specific version of `template/` as a discrete software artifact—closing the loop from research infrastructure to citable scientific contribution. Combined with Zenodo DOI registration (supported by `infrastructure.publishing`), this creates a dual-identifier citation chain: SWHID for source provenance, DOI for publication metadata [Katz et al., 2021].

1.8 Quality Assurance

1.8.1 Zero-Mock Testing Policy

The repository policy prohibits mock frameworks such as `unittest.mock`, `MagicMock`, and `patch` decorators [Martin, 2008, Meszaros, 2007]. A static gate rejects those imports, and a separate inventory surfaces semantic dependency-replacement patterns for review. Tests that require external services (Ollama or public networks) use explicit `pytest.mark` markers for conditional execution; deterministic network tests use real local HTTP servers. The philosophical motivation—analagizing excessive interaction mocking to Simmons et al.’s *researcher degrees of freedom* [Simmons et al., 2011] and the preregistration remedy [Nosek et al., 2018]—is developed fully in the **Zero-Mock Tradeoff** discussion.

The following example, drawn from this meta-project’s own suite, illustrates zero-mock compliance:

```
def test_core_module_present(self):
    # Real filesystem, real YAML parsing - no MagicMock anywhere
    modules = discover_infrastructure_modules(REPO_ROOT)
    names = [m.name for m in modules]
    assert "core" in names, f"'core' not found in {names}"
```

This test (`tests/test_meta.py::TestDiscoverInfrastructureModules::test_core_module_present`) exercises the real `discover_infrastructure_modules` function against the real repository root. There are no mock objects substituting for the directory walk, no patched YAML parsers, and no synthetic return values—the test passes only if the infrastructure modules genuinely exist and are discoverable at their expected paths.

1.8.2 Coverage Thresholds

The pipeline enforces two coverage tiers:

Tier	Scope	Minimum	Current	Rationale
Project	<code>projects/*/src/</code>	90%	Reported by the current test artifact	Domain code must be thoroughly validated
Infrastructure	<code>infrastructure/</code>	60%	Reported by the current test artifact	Broader shared surface

These thresholds are enforced at Stage 01 of the pipeline. A project test run below its declared 90% floor fails before downstream publication stages. This manuscript deliberately does not quote an achieved coverage percentage unless a fresh coverage artifact supplies it.

1.8.3 Test Suite Composition

The repository maintains three test suites:

- **Infrastructure tests** (`tests/`): about 9,557 tests validating the 28 infrastructure subdirectories, covering logging, rendering, validation, steganography, reporting, and LLM integration.
- **Project tests** (`projects/*/tests/`): Per-project suites whose sizes scale with each exemplar’s surface area — for example 300 tests in `template_autoresearch_project` and 242 in `template_code_project`, with several exemplars larger still. (A true min/max span would require dedicated `project_test_count_min/project_test_count_max` tokens in `build_manuscript_metrics_dict`; see the meta-template’s generator backlog.)
- **Integration tests**: Embedded within infrastructure tests, these exercise full pipeline stages against real manuscript inputs, validating end-to-end behavior from Markdown source to rendered PDF.

1.8.4 Visualization Standards

All generated figures must meet accessibility requirements:

- Shared 16pt font constant for primary text elements, with compact annotations derived from that constant.

- Colorblind-safe palettes (IBM Design / Wong palette) with high-contrast fallbacks.
- 200 DPI PNG rendering, sourced from the same constant used by every figure writer.
- Descriptive axis labels and figure titles.
- No reliance on color alone to convey information—redundant encoding via shape, pattern, or annotation is used where applicable.

The `test_architecture_viz.py` suite verifies that each real figure generator writes a non-empty PNG and that the comparative matrix has its declared shape and value domain. The font and render-resolution values above are injected from `viz_palette.py`, preventing prose and implementation from drifting even though visual accessibility still requires human inspection of rendered figures.

2 Discussion

2.1 The Zero-Mock Tradeoff

The **Zero-Mock testing policy** is `template/`'s most distinctive design decision. By prohibiting all mock objects, we gain confidence that tests exercise real code paths—a `pytest` run against the template genuinely invokes `pandoc`, writes to disk, and parses real YAML. The cost is test duration: the full infrastructure test suite (about 9,557 tests) runs for minutes rather than the sub-second execution typical of heavily-mocked suites. This manuscript deliberately reports no wall-clock figure, consistent with the Results section's discipline of declining timing claims without a versioned benchmark artifact.

We argue this tradeoff is strongly favorable for research software. Unlike web applications where millisecond latency and thousands of daily deploys demand fast feedback loops, research pipelines run infrequently (once per manuscript revision) and correctness vastly outweighs speed. A mocked test that passes while the real renderer fails is worse than a slow test that catches the failure. The analogy to statistical methodology is precise: just as Simmons et al.'s *researcher degrees of freedom* [Simmons et al., 2011] inflate false-positive rates through undisclosed analytical flexibility, mock objects create *testing degrees of freedom* that make integration failures invisible. The Zero-Mock policy closes this loophole by the same mechanism that pre-registration [Nosek et al., 2018] closes the p-hacking loophole: removing flexibility before the fact. As Peng [Peng, 2011] argues, computational reproducibility requires independent verification—and mock-only tests verify assumptions rather than results. Garijo et al.'s FAIRsoft evaluator [Garijo et al., 2024] identifies *executability* as a primary quality indicator; the Zero-Mock policy operationalizes executability at the unit level.

2.1.1 When Mocks Are Not the Problem

It is important to distinguish the Zero-Mock policy from a naive rejection of all test isolation techniques. Fowler's classification [Martin, 2008] recognizes that stubs and fakes serve legitimate purposes—a test database populated with known data is not a mock, it is a fixture. The policy specifically prohibits *mock objects* as defined by Meszaros: assertions on indirect outputs (method calls, argument patterns) rather than direct outputs (return values, side effects). The distinction matters because mock-based assertions encode implementation assumptions ("method X must be called with argument Y") that become invisible coupling between tests and production code, creating the illusion of coverage without testing real behavior.

2.1.2 Practical Implementation

The template enforces zero-mock compliance at three levels:

1. **Code review:** `AGENTS.md` at every directory level explicitly states the prohibition, ensuring both human and AI contributors are aware before writing tests.
2. **Static analysis:** `scripts/audit/verify_no_mock.py` implements the enforced lexical gate (prohibited mock-framework imports and calls) and an `--inventory` mode that separately classifies environment isolation from dependency-replacement debt; pre-push runs it automatically.
3. **Cultural norm:** `template_code_project` documents filesystem + YAML + plotting paths while `template_autoresearch_project` exercises readiness planning; `template_search_project` reinforces HTTP-realistic literature queries—both serve as onboarding references alongside infra suites.

However, the policy requires careful management of external dependencies. Tests requiring Ollama (the local LLM backend) use `@pytest.mark.requires_ollama` and are skipped in environments where the service is unavailable. Tests requiring network access use `@pytest.mark.network`. This marker system preserves the Zero-Mock principle while acknowledging that not all environments provide all services, especially computationally intensive ones. The key distinction is between *replacing* an external dependency (which mock objects do, hiding failures) and *skipping* a test when a dependency is absent (which markers do, preserving transparency).

2.2 Comparison to Existing Tools

The [gap analysis](#) established that no single tool integrates all six cross-cutting concerns. Here we synthesize the [fourteen-dimension comparison](#) into three structural insights. First, the landscape bifurcates: workflow managers (Snakemake [[Köster and Rahmann, 2012](#)], Nextflow [[Di Tommaso et al., 2017](#)], CWL [[Amstutz et al., 2016](#)]) provide distributed execution but no manuscript support; publication tools (Quarto [[Allaire et al., 2024](#)], Jupyter Book, R Markdown [[Xie et al., 2018](#)], Overleaf [[Overleaf \(Digital Science\), 2025](#)], Prism [[OpenAI, 2026](#)]) author documents but embed no integrity guarantees; and DVC [[Iterative, Inc., 2024](#)] versions artifacts without orchestrating pipelines. `template/` occupies the intersection, sacrificing distributed execution for unified enforcement of testing, provenance, and documentation. This positioning is complementary—a mature deployment might use Nextflow upstream and `template/` for rendering, testing enforcement, and provenance downstream. Typst [[Mädje and Haug, 2023](#)], with its faster compilation cycle, is not one of the nine compared peers but could serve as an alternative rendering backend if a Pandoc writer were contributed.

Second, the eight enforcement capabilities `template/` co-locates—testing enforcement, coverage thresholds, steganographic watermarking, multi-project management, AI-agent documentation, the agentic skill protocol, an interactive TUI, and Zero-Mock policy—are individually straightforward (and several, such as multi-project management and AI-agent documentation, are matched in part by individual peers). Their value here lies in a shared pipeline contract: required test and documentation gates run together, while selected provenance stages carry their own validation. The FAIR4RS principles [[Barker et al., 2022](#), [Lamprecht et al., 2020](#)] articulate what research software quality requires; FAIRsoft [[Garijo et al., 2024](#)] scores compliance observationally; `template/` operationalizes part of this agenda by failing project tests below the declared 90% floor and validating provenance artifacts when those stages are enabled. Cohen et al.’s four pillars of research software engineering [[Cohen et al., 2021](#)]*—*sustainability, quality, community, and policy*—*are addressed primarily through the first two pillars via quality-gated automation.

Third, the AI-agent documentation dimension reveals an underserved need. Overleaf and Prism provide AI *writing* assistance, but neither exposes structured documentation for *external* agents to consume. `template/`’s AGENTS.md + SKILL.md layer enables an agent entering the repository to discover capabilities, understand API contracts, and invoke modules without prior training ([Documentation Duality, AI Collaboration](#)).

2.2.1 FAIR4RS Evolution (2024–2026)

Since the FAIR4RS principles were published [[Barker et al., 2022](#)], the community has moved toward operationalization. The RDA Virtual Plenary 24 (April 2025) featured a two-year retrospective review [[Honeyman et al., 2024](#)] recommending principle amendments—notably adding *reproducibility* as an explicit requirement and clarifying “domain-relevant standards”—alongside a leadership refresh and parallel guidance activities. The ReSA Actionable FAIR4RS Task Force (launched December 2024) analyzed the 17 principles into six actionable categories (identifiers, metadata for publication/discovery/reuse, standards, references, and licenses), with a first draft expected by September 2025 [[Research Software Alliance, 2024](#)]. Tools for automated FAIR assessment have also matured: the F-UJI extension for research software evaluation now scores against FRSM-04 through FRSM-17 metrics, complementing Garijo et al.’s FAIRsoft evaluator [[Garijo et al., 2024](#)]. `template/`’s pipeline-enforced quality gates—coverage thresholds, documentation completeness checks, and provenance embedding—anticipate this operationalization trend by implementing FAIR4RS not as a post-hoc assessment but as an architectural invariant.

In Gentleman and Temple Lang’s terminology [[Gentleman and Temple Lang, 2007](#)], `template/` is a *research compendium* scaled to the repository level—bundling not just one study’s code and data but N studies, with shared infrastructure, automated testing, and embedded provenance. Nüst et al.’s executable research compendium (ERC) [[Nüst et al., 2017](#)] extends this vision with containerized reproduction environments; `template/` complements containerization by adding the testing enforcement, multi-project management, and provenance embedding layers that ERCs do not address.

2.3 The AI Collaboration Model

The **Documentation Duality** standard and three-tier skill architecture represent an empirical bet: that structured, machine-readable documentation measurably improves AI agent performance in research codebases. This section reports our key observations.

The documentation investment creates a positive feedback loop: as agents produce higher-quality outputs from structured context, developers maintain that documentation, which in turn improves future interactions [Lau and Guo, 2025]. We observed this concretely during `template/` development—each module’s `SKILL.md` was refined through iterative AI-assisted generation, serving as both input prompt and output validator.

The `SKILL.md` layer, with its MCP-aligned YAML frontmatter [Anthropic, 2024], provides a bridge to the agentic software paradigm. Lu et al.’s AI Scientist [Lu et al., 2024] demonstrates end-to-end autonomous research, while OpenHands is evaluated on SWE-Bench Verified [Wang et al., 2024b, Jimenez et al., 2024]. These systems motivate structured, protocol-aligned tool inventories for unfamiliar codebases. An agent navigating `template/` reads `CLAUDE.md` for global constraints, scans `AGENTS.md` for local contracts, and can invoke skill-enabled capabilities through `SKILL.md` descriptors. The live module inventory verifies `AGENTS.md` and `README.md` coverage; `SKILL.md` remains a capability-specific layer rather than a universal claim about every infrastructure directory.

This three-tier model is, to our knowledge, novel in the research software engineering literature. The scale of the investment is substantial: 404 Markdown files under `docs/` alone, plus an `AGENTS.md/README.md` pair in every directory and a `SKILL.md` descriptor on skill-enabled infrastructure modules. That count is itself injected from live introspection—the manuscript refuses to quote a documentation total it cannot recompute—and it represents a deliberate commitment to machine-readable context that shrinks the surface on which an agent can hallucinate.

2.4 The Learning Curve

The Thin Orchestrator pattern imposes a cognitive overhead on researchers accustomed to writing monolithic scripts. The requirement to factor all logic into `src/` modules and use scripts only as stateless wiring introduces an additional layer of indirection. We mitigate this through:

1. **Template exemplars:** `template_code_project` ships minimal optimization commentary; `template_prose_project` and `template_autoresearch_project` broaden narrative + retrieval scaffolding.
2. **Documentation Duality:** Every directory has both `README.md` (for humans) and `AGENTS.md` (for AI collaborators), reducing the cost of navigation.
3. **Interactive orchestrator:** `run.sh` provides a TUI menu that abstracts pipeline complexity.
4. **Skill-level documentation:** The `docs/guides/` directory provides a progressive sequence of beginner, intermediate, advanced, and expert guides alongside a comprehensive new-project setup checklist.

2.5 Limitations

- **LaTeX dependency:** The rendering pipeline requires a full TeX distribution (TeX Live or MiKTeX), which is a 4–6 GB install. This is the largest single dependency and is a barrier for researchers without system-level package management access.
- **Python-centric:** The infrastructure layer is Python-only. Projects in other languages can use the rendering and steganography stages but cannot leverage the `scientific` or `validation` modules.
- **Single-machine:** The pipeline runs locally. Distributed execution (e.g., across a compute cluster) is not natively supported, a gap where Snakemake, Nextflow, and CWL have clear superiority.
- **Steganographic robustness:** Alpha-channel overlays are stripped by aggressive PDF optimization tools (e.g., `qpdf --optimize`). QR codes are visible and removable. The current system provides *tamper detection* (via SHA-256 hashing) but not *non-repudiation* in the cryptographic sense—it lacks private-key digital signatures. An attacker with access to the source code could reproduce the watermark without having run the original pipeline.
- **Test duration:** The Zero-Mock policy increases test execution time from sub-second (mocked) to multi-minute (real) for the full infrastructure suite. This is acceptable for research workflows but may not suit continuous deployment scenarios.

- **AI-native writing tools:** `template/` does not include an integrated AI writing assistant comparable to Overleaf's Copilot features or OpenAI Prism's GPT-5.2 context-aware editing. The `infrastructure.llm` module provides LLM review as a pipeline stage but not as an interactive writing environment.

2.6 Future Directions

1. **Supply-chain provenance:** Integration with software supply chain frameworks such as in-toto [Torres-Arias et al., 2019] and SLSA (Supply-chain Levels for Software Artifacts) [Open Source Security Foundation, 2023] to provide end-to-end attestation from source commit through build pipeline to published artifact. SLSA’s four graduated levels of build integrity (from basic provenance to hermetic, reproducible builds) provide a natural extension ladder for the template’s currently document-centric provenance model. The template’s existing steganographic layer embeds document-level provenance; supply-chain frameworks would add build-level provenance, closing the gap between “this PDF was produced by this pipeline” and “this pipeline was executed with this verified source code.”
2. **Decentralized provenance:** Integration with IPFS or Arweave for immutable publication records, extending the SHA-256-based tamper detection to content-addressed storage networks.
3. **Digital signatures:** GPG or X.509 signing integrated into the steganographic layer, providing cryptographic non-repudiation in addition to tamper detection.
4. **Continuous integration:** GitHub Actions workflows that execute the pipeline on every push, with PDF artifacts as release assets and automated DOI registration via Zenodo.
5. **Multi-language support:** Extension of the Thin Orchestrator pattern to R, Julia, and Rust projects, enabling polyglot research workflows within the Two-Layer Architecture.
6. **Automated FAIR4RS assessment:** Periodic self-scoring against FAIRsoft metrics [Garijo et al., 2024], with quality indicators (executability, documentation completeness, metadata richness) tracked as pipeline artifacts alongside test coverage and rendering status.
7. **Knowledge graph integration:** Connecting pipeline outputs to Active Inference Knowledge Base entries for automated meta-analysis and cross-project citation tracking.
8. **Formal verification:** Static analysis tooling to enforce the Thin Orchestrator pattern—verifying that scripts contain no algorithmic logic and that `src/` modules do not import from `scripts/`.
9. **Agentic research pipelines via MCP:** The SKILL.md descriptors already define the interface contracts for each infrastructure module; the natural next step is to expose them as MCP server endpoints [Anthropic, 2024]. An MCP server wrapping `infrastructure.llm` would expose `query`, `review_manuscript`, and `translate_abstract` as protocol-native Tools; an MCP server wrapping `infrastructure.publishing` would expose `publish_to_zenodo` and `generate_citation_bibtex`. A research agent could then compose these Tools to execute the full pipeline—environment setup → test execution → analysis → rendering → validation → LLM review → DOI registration—without any human in the loop. This closes the loop opened by Lu et al.’s AI Scientist [Lu et al., 2024], which demonstrated automated hypothesis generation and experimental iteration but relied on ad hoc laboratory scaffolding. `template/`’s pipeline, fully exposed as MCP tools, provides that scaffolding in a reproducible, versioned, and certified form. Longer-term, the Agent2Agent (A2A) protocol [Google, 2025] enables heterogeneous specialist agents—a statistical analyst, a figure designer, a peer-review simulator—to coordinate via a shared, protocol-mediated workspace built on precisely the kind of modular, well-documented infrastructure that `template/` provides.
10. **Research Infrastructure as Code and software citation:** The DevOps IaC paradigm, applied to research, means the entire manuscript pipeline is a version-controlled, reproducible artifact in its own right. Software Heritage [Di Cosmo et al., 2020] provides persistent SWHIDs (Software Hash Identifiers) for source-code snapshots, enabling `template/` itself to be cited as a software artifact with a DOI-equivalent stable identifier. Combining this with Zenodo DOI registration (already supported by `infrastructure.publishing`) creates a full citation chain: the paper cites the data (DOI), the data provenance cites the pipeline (SWHID), and the pipeline cites the framework release (Zenodo DOI). This three-link citation chain operationalizes the Katz et al. [Katz et al., 2021] software citation principles at the infrastructure level.

2.7 Conclusion

`template/` demonstrates that high-integrity, reproducible research need not be onerous. By embedding provenance, testing, and documentation into the architecture itself—rather than layering them atop a fragmented workflow—the template transforms “best practices” from aspirational guidelines into enforced invariants [Wilson et al., 2017, Sandve et al., 2013, Lamprecht et al., 2020]. The Two-Layer Architecture

ensures that infrastructure improvements propagate to all projects without coupling. The Zero-Mock policy ensures that tests **reflect reality**. The steganographic provenance layer ensures that published artifacts carry their own **authentication**. The **comparative analysis** confirms that no existing tool integrates all eleven distinctive capabilities—testing enforcement, coverage thresholds, cryptographic provenance, steganographic watermarking, multi-project management, AI-agent documentation, the agentic skill protocol, interactive TUI, Zero-Mock policy, manuscript rendering, and pipeline orchestration—within a single enforced pipeline. The template is not merely a build tool; it is an epistemological commitment. It asserts that a research paper is not a static document but a build artifact—reproducible, verifiable, and traceable to the code that generated it. As Knuth observed, programs should be written for humans to read and only incidentally for machines to execute [Knuth, 1984]. We extend this dictum: research manuscripts should be *built* for verification and only incidentally for reading. In an era of generative AI, AI-native research workspaces, and synthetic media—where the boundary between human-authored and machine-generated text grows increasingly indeterminate [Gruenpeter et al., 2021]—the provenance chain from source code to published PDF is not an administrative convenience. It is the epistemic ground on which scientific trust must be rebuilt. That this manuscript was itself built, tested, and watermarked by the pipeline it describes—its metrics computed from the repository it inhabits, its figures rendered by the code it documents—is not a rhetorical device but a structural proof: the system works because you are reading its output.

3 Infrastructure Module Reference

This section inventories every Layer-1 subdirectory returned by `28 discover_infrastructure_modules(repo_root)`. File totals use 708 Python sources across infra + 9,557 infra tests guarding them. Documentation Duality = paired `README.md` + `AGENTS.md`; optional `SKILL.md` manifests feed `python -m infrastructure .skills`.

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
autoresearch	10	✓	✓	<code>build_autoresearch_plan</code> , readiness validation CLI
benchmark	4	✓	✓	Template harness scoring + comparative gates
config	0	✓	✓	Repository defaults + hardened templates
core	120	✓	✓	<code>get_logger</code> , <code>load_config</code> , <code>TemplateError</code>
docker	0	✓	✓	Containerisation scaffolding
doctor	14	✓	✓	Checkout diagnose/fix/undo repairs
documentation	14	✓	✓	<code>FigureManager</code> , <code>generate_glossary</code>
fonds	6	✓	✓	—
llm	55	✓	✓	Ollama helpers, sanitization, review + translation
logrotate.d	0	✓	✓	pipelines Rotation snippets (documentation-first)
methods	5	✓	✓	<code>build_methods_orchestration_plan</code> , methods-stage contracts + validation
orchestration	12	✓	✓	<code>PipelineRunner</code> , entry point for <code>./run.sh</code>
project	41	✓	✓	<code>discover_projects</code> , workspace management
prose	9	✓	✓	Markdown readability + prose tooling
provenance	7	✓	✓	—
publishing	81	✓	✓	Zenodo, executable bundle, archival targets

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
reference	16	✓	✓	BibTeX models, parsers, converters
rendering	65	✓	✓	PDF/HTML/slide rendering, Pandoc filters
reporting	57	✓	✓	Coverage parsers, dashboards, executive artefacts
research	3	✓	✓	—
rules	6	✓	✓	—
scientific	4	✓	✓	check_numerical_ stability, benchm ark_function
search	62	✓	✓	infrastructure.s earch.literature clients + cache
sia	10	✓	✓	Self-Improving-AI loop: task validation, harness, metric capture
skills	8	✓	✓	discover_skills, SKILL manifest regeneration
steganography	13	✓	✓	Watermark overlays + hash manifests
tools	6	✓	✓	—
validation	80	✓	✓	PDF + Markdown + integrity CLIs

3.1 Alphabetical summaries

Below, `${module}_python_file_count` placeholders expand per subdirectory at render-time.

3.1.1 `infrastructure.autoresearch` (10 files)

Readiness planner, validation CLI, and report models for AutoResearch-style project promotion (`infrastructure/autoresearch/`).

3.1.2 `infrastructure.benchmark` (4 files)

Template harness scoring and comparative gate helpers exercised in CI smoke paths.

3.1.3 `infrastructure/config` (non-package subdirectory)

Repository-wide YAML templates and secure manifests (`.env.template`, hardened defaults referenced by Docker + CLI). `config/` carries no `__init__.py`, so it is a configuration subdirectory rather than an importable package.

3.1.4 `infrastructure.core` (120 files)

Checkpointing, logging, pipeline YAML parsing, telemetry bridges, filesystem helpers, hardened exceptions. Everything else imports logging + error taxonomy from here first.

3.1.5 infrastructure.doctor (14 files)

Checkout diagnose/fix/undo repairs for broken local workspace states.

3.1.6 infrastructure.docker (0 files)

Pinned images / compose scaffolding for reproducible CI + remote builds.

3.1.7 infrastructure.documentation (14 files)

Figure registries plus glossary tooling feeding manuscript automation.

3.1.8 infrastructure.fonds (6 files)

Resource pool management for curated fonds (tracked reference datasets, bibliographic collections, and evidence corpora). Fonds mirror `projects/templates/` with git-tracked `templates/*` exemplars and sidecar-linked private lifecycle folders.

3.1.9 infrastructure.llm (55 files)

Ollama integrations, sanitization adapters, templated reviewer flows. **Literature ingestion now lives primarily in `search/literature` + `citation` helpers in `reference/`.**

3.1.10 infrastructure.methods (5 files)

Deterministic methods-orchestration contracts (`MethodStage`, `MethodsOrchestrationPlan`, `MethodsIssue`): builds and validates an ordered methods plan for a research project so the manuscript's "Methods" track stays bound to executable stages.

3.1.11 infrastructure.orchestration (12 files)

`python -m infrastructure.orchestration` exposes interactive menus, subprocess wiring for thin shell wrappers (`run.sh`, `secure_run.sh`), and stubs used in CI for menu parsing tests.

3.1.12 infrastructure.project (41 files)

Canonical discovery (`discover_projects`) enforcing `src/` + `tests/`, slug validation, nested WIP namespaces.

3.1.13 infrastructure.prose (9 files)

Readability metrics + Markdown tooling for prose-centric manuscripts / CI gates.

3.1.14 infrastructure.provenance (7 files)

Content-addressed provenance DAG. Records artifact lineage (which run produced which file, from which inputs) as a verifiable graph of artifact/run/source/claim nodes connected by produced/consumed/derived-from/supports/refutes edges. Includes a structured Review system with severity (blocking/major/minor/info) and verdict (refutes/supports). Features a CLI and pipeline integration hooks for automatic lineage recording after every stage.

3.1.15 infrastructure.publishing (81 files)

Metadata models, APA/BibTeX/MLA formatters, optional Zenodo clients.

3.1.16 infrastructure.reference (16 files)

Citation/BibTeX parsing + conversion utilities leveraged by manuscripts and retrieval scripts.

3.1.17 `infrastructure.rendering` (65 files)

Pandoc shim, Unicode/XeLaTeX postprocessors, combined PDF/HTML/slide exporters.

3.1.18 `infrastructure.reporting` (57 files)

Parses pytest + coverage artefacts for dashboards; pairs with Stage 01 summaries and downstream executive exports.

3.1.19 `infrastructure.research` (3 files)

Seven-stage research workflow (SCOPE→LITERATURE→REASON→DESIGN→COMPUTE→SYNTHESIZE→WRITE) defined as typed ResearchStage data classes with explicit outputs, skills, and template commands per stage. Includes a full PRISMA-adapted literature review prompt and a research workflow prompt referencing template/ infrastructure commands.

3.1.20 `infrastructure.rules` (6 files)

Governance rules layer for validating project lifecycle transitions, sidecar sync policies, and pipeline gate orchestration. Rules mirror `projects/templates/` with git-tracked `templates/*` exemplars.

3.1.21 `infrastructure.scientific` (4 files)

Stability probing, benchmarking hooks—consumed heavily by optimization exemplars (`template_code_project` scripts).

3.1.22 `infrastructure.search` (62 files)

Two-tier search architecture: the `literature/` client stack (`client.py`, backends, caches) powers literature search with arXiv, Crossref, local, and Paperclip backends. `connectors/` exposes the built-in scientific database adapters through a uniform `ConnectorRegistry`; OpenAlex, UniProt, PDB, Semantic Scholar, European PMC, bioRxiv, and other registered adapters share list/search CLI commands with HTTP timeout, retry, and TTL caching. The live registry, not this prose, is authoritative for connector count.

3.1.23 `infrastructure.sia` (10 files)

Generic Self-Improving-AI loop utilities: task-layout validation, execution harness, and metric capture reused by `template_sia` (fixture-replay by default).

3.1.24 `infrastructure.skills` (8 files)

Discovers SKILL.md frontmatter → `.cursor/skill_manifest.json`.

3.1.25 `infrastructure.steganography` (13 files)

Watermark overlays, hashing companions triggered by secure pipeline path.

3.1.26 `infrastructure.tools` (6 files)

Invocable tool definitions registered by resource-pool governance; tools mirror `projects/templates/` with git-tracked `templates/*` exemplars.

3.1.27 `infrastructure.validation` (80 files)

Markdown + PDF + integrity CLIs underpinning Stage 04 diagnostics.

3.1.28 `infrastructure/logrotate.d` (0 files)

Operational templates for deployments (documentation-first; intentionally minimal Python footprint).

Documentation maturity: Coverage statements in Results pull from introspection—not hand-maintained denominators—so newly promoted modules automatically flow into manuscripts after `generate_manuscript_metrics.py`.

FAIR+RSE linkage: MCP-ready SKILL.md artefacts align with evaluator heuristics (executability + meta-data richness) emphasized by FAIRsoft guidance [Garijo et al., 2024].

4 Appendices

4.1 Appendix: Pipeline Stage Reference

Table 1: Single-project DAG exported from default `pipeline.yaml` (names shown in topological order). Scripts live under `scripts/`.

Stage name	Script / method	Primary inputs	Outputs / artefacts	Failure mode
Clean Output Directories	<code>_run_clean_outputs</code>	prior <code>projects/<name>/output/</code> , mirrored <code>output/<name>/targets</code>	emptied trees	Blocking
Environment Setup	<code>scripts/pipeline/stage_00_setup.py</code>	toolchain probes	scaffold dirs, env exports	Blocking
Infrastructure Tests	<code>scripts/pipeline/stage_01_test.py</code> <code>--infra-only --infra-scope pipeline-smoke</code>	<code>tests/infra_tests/</code>	coverage + junit-style logs	tolerant ceilings
Project Tests	<code>scripts/pipeline/stage_01_test.py</code> <code>--project-only</code>	<code>projects/<name>/tests/</code>	stage artefacts	blocking by default
Project Analysis	<code>scripts/pipeline/stage_02_analysis.py</code>	thin scripts	<code>figures/</code> , <code>data/</code> , reports	Blocking
Connector Search	<code>scripts/pipeline/stage_08_connector_search.py</code>	<code>manuscript/config.yaml</code>	<code>output/data/connector_search/</code>	opt-in (science tag); skipped if not configured
Provenance Record	<code>scripts/pipeline/stage_09_provenance_record.py</code> <code>--stage "Connector Search"</code>	prior stage outputs	<code>.provenance/graph.json</code>	opt-in (provenance tag); skipped if not configured
PDF Rendering	<code>scripts/pipeline/stage_03_render.py</code>	<code>manuscript/</code> , placeholders	<code>.pdf/</code> <code>.tex</code> bundles	Blocking
Output Validation	<code>scripts/pipeline/stage_04_validate.py</code>	render tree	Markdown + PDF diagnostics JSON	Blocking / downgrade
LLM Scientific Review	<code>scripts/pipeline/stage_06_llm_review.py</code> <code>--reviews-only</code>	resolved manuscript artefacts	textual reviews	Optional skip (<code>allow_skip</code>)
LLM Translations	<code>scripts/pipeline/stage_06_llm_review.py</code> <code>--translations-only</code>	abstract metadata	multilingual snippets	Optional skip (<code>allow_skip</code>)
Copy Outputs	<code>scripts/pipeline/stage_05_copy.py</code>	validated tree	mirrored <code>output/<name>/...</code>	soft fail logged

Stage name	Script / method	Primary inputs	Outputs / artefacts	Failure mode
Ebook Generation	<code>scripts/pipeline/stage_11_ebook.py</code>	rendered combined markdown	<code>output/ebook/</code> (EPUB/MOBI/DOCX)	opt-in (ebook tag); soft fail
Metadata Package	<code>scripts/pipeline/stage_12_metadata.py</code>	manuscript/conf ig.yaml	<code>output/metadata/</code> (ONIX/JSON/OPF)	opt-in (metadata tag); soft fail
Executable Bundle	<code>scripts/runner/bundle_executable.py</code>	project tree + outputs	container bundle manifest	opt-in (bundle tag)
Archival Publication	<code>scripts/runner/archive_publication.py</code>	bundle + deliverables	archival deposit manifest	opt-in (archival tag)

`scripts/pipeline/stage_07_executive_report.py` is invoked **outside** this DAG whenever `execute_multi_project.py` aggregates pipelines—supplying cross-project KPI dashboards absent from lone-project checkpoints.

4.2 Appendix: Configuration Reference

Table 2: Configuration schema for `config.yaml`, showing all supported fields and their structure.

```
paper:
  title: "Paper Title"
  subtitle: "Optional Subtitle"
  version: "1.0"
  date: "2026-03-19"

authors:
  - name: "Author Name"
    orcid: "0000-0000-0000-0000"
    email: "author@example.com"
    affiliation: "Institution"
    corresponding: true

publication:
  doi: "10.5281/zenodo.XXXXXX"
  version_doi: "10.5281/zenodo.XXXXXX"
  version_record: "https://zenodo.org/records/XXXXXX"
  github_repository: "owner/repository"
  repository_url: "https://github.com/owner/repository"
  published_artifacts:
    github_pages: "https://owner.github.io/repository/"
    software_heritage: "https://archive.softwareheritage.org/..."
    pypi: "https://test.pypi.org/project/..."
    osf: "https://osf.io/..."
    huggingface_hub: "https://huggingface.co/datasets/owner/repository"
    netlify: "https://..."
    ipfs_pinata: "https://gateway.pinata.cloud/ipfs/..."
  transmission_bookends:
    enabled: false
    max_prior_releases: 5
    show_steganography: true
  journal: "Target Journal"
  volume: "1"
  pages: "1-10"
  year: "2026"

keywords:
  - "keyword1"
  - "keyword2"

metadata:
  license: "Apache License 2.0"
  language: "en"

llm:
  reviews:
    enabled: true
    types: [executive_summary, quality_review]
  translations:
```

```
    enabled: false

testing:
  max_test_failures: 0
  max_infra_test_failures: 3
  max_project_test_failures: 0
```

4.3 Appendix: Repository Directory Structure

```
template/
  infrastructure/
    config/ docker/ documentation/ llm/
    orchestration/    # Thin Python entry equal to `./run.sh` backend
    prose/ reference/ rendering/ reporting/
    scientific/ search/ skills/ steganography/ validation/
    project/ core/
    logrotate.d/      # Operational rotation templates (no Python pkg)
  scripts/
    pipeline/          # stage_00_setup.py ... stage_12_metadata.py (canonical, cited by pipeline.yaml)
    runner/            # execute_pipeline.py execute_multi_project.py bundle_executable.py archive_pu
  projects/            # Typed program subfolders (`discover_projects`)
  templates/           # Public exemplars (git-tracked) - full roster:
                        # docs/_generated/active_projects.md (regenerated by scripts/docgen/ac
                        template_template/ # Present manuscript (`manuscript/` here)
  active/              # Hot-seat rendered set (symlinked, private)
  working/             # Non-rendered backburner (symlinked, private)
  published/           # Non-rendered published (symlinked, private)
  archive/             # Non-rendered retired (symlinked, private)
  other/              # Non-rendered misc (symlinked, private)
docs/ (18 top-level areas, 404+ markdown files per live counter)
tests/                # Infra suites (557+ files)
AGENTS.md / README.md / CLAUDE.md / pyproject.toml
run.sh / secure_run.sh
output/ ...           # Mirrors after copy stage
See docs/_generated/active_projects.md for regenerated slugs (uv run python scripts/docgen/acti
ve_projects.py).
```

4.4 Appendix: Documentation Inventory

The repository maintains documentation at three levels:

Table 3: Documentation inventory across the four-layer documentation architecture, from repository-wide system files to per-module skill descriptors.

Level	Files	Purpose
Repository root	AGENTS.md, CLAUDE.md, README.md, RUN_GUIDE.md	Global navigation and AI agent context
docs/ directory	404 files across 18 subdirectories	User guides, API reference, troubleshooting
Per-directory	AGENTS.md + README.md at every directory	Documentation Duality standard
Per-module (Tier 3)	SKILL.md at every infrastructure module	Machine-parseable MCP-aligned skill descriptor
Infrastructure-level (PAI)	PAI.md at infrastructure/ directory	Personal AI Infrastructure integration contract

The `docs/` subdirectories cover: `core/` (essential docs), `guides/` (progressive skill-level guides), `architecture/` (system design), `usage/` (content authoring), `operational/` (build, config, logging, troubleshooting), `reference/` (API, FAQ, glossary), `modules/` (28 infrastructure modules), `development/` (contributing, testing), `maintenance/` (long-horizon maintenance: private projects, CI-local, regression, archival, bundle), `best-practices/` (version control, migration), `plans/` (strategic plans and decision records), `rules/` (contributor norms), `prompts/` (22 AI prompt templates), `security/` (steganography, hashing), `streams/` (livestream notes), and `audit/` (review reports), plus the machine-generated `_generated/` snippets and `images/` assets.

Every count in this appendix is injected from live repository introspection rather than hand-maintained: 404 counts every Markdown file beneath `docs/` recursively, 18 counts its first-level subdirectories, and 22 counts the workflow subdirectories that each carry a `SKILL.md` descriptor. This is the same discipline the manuscript argues for throughout—a hand-typed documentation total silently rots as the tree grows, whereas a token re-resolves on every render. A reader onboarding to the repository should start at `docs/core/`, follow the graduated `docs/guides/` skill ladder, and consult the per-directory `AGENTS.md/README.md` pair nearest to whatever code they are editing; AI agents additionally read each module’s `SKILL.md` to locate capabilities without guessing API signatures.

4.5 Appendix: Comparative Tool Matrix

Symbol key (applies to all cells): **Y** = full native support · **about** = partial or plugin-based · **N** = absent. See also [Figure 4](#) for a colour-coded heatmap rendering of this table.

Table 4: Comparative feature matrix (14 capabilities $\times$ 10 tools). Y = full native support, about = partial or plugin-based, N = absent.

Capability	template	Snakemake	Nextflow	CWL	Quarto	Jupyter Book	R Mark-down	DVC	Overleaf	OpenAI Prism
Pipeline orchestration	Y	Y	Y	Y	about	N	N	Y	N	N
Manuscript rendering	Y	N	N	N	Y	Y	Y	N	Y	Y
Testing enforcement	Y	N	N	N	N	N	N	N	N	N
Coverage thresholds	Y	N	N	N	N	N	N	N	N	N
Cryptographic provenance	Y	N	about ₁	N	N	N	N	about ₂	N	N
Steganographic watermarking	Y	N	N	N	N	N	N	N	N	N
Multi-project management	Y	N	N	N	N	N	N	N	about	about
AI-agent documentation	Y	N	N	N	N	N	N	N	about	about
Agentic skill protocol (SKILL.md / MCP)	Y	N	N	N	N	N	N	N	N	N
Interactive TUI	Y	N	N	N	N	N	N	N	N	N
Zero-mock policy	Y	N	N	N	N	N	N	N	N	N
Container support	about	Y	Y	Y	N	N	N	N	N	N
Distributed execution	N	Y	Y	Y	N	N	N	about ₃	N	N
Multi-language (R/Julia)	N	Y	N	Y	Y	Y	Y	Y	N	N

¹ Nextflow lineage records operate at workflow scope rather than as a rendered-document citation graph. ² DVC provides content-addressed versioning for data artifacts via its object store. ³ DVC integrates with

remote storage (S3, GCS, Azure) but does not natively orchestrate distributed compute. This matrix is a versioned manuscript snapshot, not a continuously updated product survey. A future refresh should re-check every external capability against current primary documentation before changing a cell.

References

- J. J. Allaire, Charles Teague, Carlos Scheidegger, Yihui Xie, and Christophe Dervieux. Quarto, 2024. URL <https://github.com/quarto-dev/quarto-cli>.
- Peter Amstutz, Michael R. Crusoe, Nebojša Tijanić, Brad Chapman, John Chilton, Michael Heber, Andrey Kartashov, Dan Leehr, Hervé Ménager, Maya Nedeljkovich, Matt Scales, Stian Soiland-Reyes, and Luka Stojanovic. Common workflow language, v1.0. *Specification, Common Workflow Language working group*, 2016. doi: 10.6084/m9.figshare.3115156.v2.
- Anthropic. Introducing the model context protocol. Anthropic Engineering Blog, November 2024. URL <https://www.anthropic.com/news/model-context-protocol>. Open-source protocol for secure, two-way communication between AI models and external tools/data sources. Donated to the Linux Foundation, December 2025.
- Michelle Barker, Neil P. Chue Hong, Daniel S. Katz, Anna-Lena Lamprecht, Carlos Martinez-Ortiz, Fotis Psothopoulos, Jennifer Harrow, Leyla Jael Castro, Morane Gruenpeter, Paula Andrea Martinez, and Tom Honeyman. Introducing the FAIR principles for research software. *Scientific Data*, 9(1):622, 2022. doi: 10.1038/s41597-022-01710-x.
- Jeremy Cohen, Daniel S. Katz, Michelle Barker, Neil P. Chue Hong, Robert Haines, and Caroline Jay. The four pillars of research software engineering. *IEEE Software*, 38(1):97–105, 2021. doi: 10.1109/MS.2020.2973362.
- Roberto Di Cosmo, Morane Gruenpeter, and Stefano Zacchiroli. Referencing source code artifacts: A separate concern in software citation. *Computing in Science & Engineering*, 22(2):33–43, 2020. doi: 10.1109/MCSE.2019.2963148.
- Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4):316–319, 2017. doi: 10.1038/nbt.3820.
- Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995. ISBN 0-201-63361-2.
- Daniel Garijo, Luis García, Matthias Barker, Rubén Chaves, Oscar Corcho, et al. FAIRsoft—a practical implementation of FAIR principles for research software. *Bioinformatics*, 40(8):btae464, 2024. doi: 10.1093/bioinformatics/btae464.
- Robert Gentleman and Duncan Temple Lang. Statistical analyses and reproducible research. *Journal of Computational and Graphical Statistics*, 16(1):1–23, 2007. doi: 10.1198/106186007X178663.
- Google. Agent2agent (A2A) protocol specification. Open Protocol Specification, 2025. URL <https://a2a-protocol.org/latest/specification/>. Open protocol for AI agent interoperability built on HTTP, JSON-RPC, and SSE; announced at Google Cloud Next '25, April 2025.
- Morane Gruenpeter, Daniel S. Katz, Anna-Lena Lamprecht, Tom Honeyman, Daniel Garijo, Alexander Struck, Anna Niehues, Paula Andrea Martinez, Leyla Jael Castro, Tovo Rabemanantsoa, Neil P. Chue Hong, Carlos Martinez-Ortiz, Laurents Sesink, Matthias Liffers, et al. Defining research software: A controversial discussion. In *SSRN Preprint*, 2021. doi: 10.2139/ssrn.3884311.
- Tom Honeyman, Michelle Barker, Daniel S. Katz, and Neil P. Chue Hong. The FAIR principles for research software: Three years on. Research Data Alliance Virtual Plenary 24, 2024. URL https://www.rd-alliance.org/wp-content/uploads/2025/04/RDA_VP24_FAIR4RS_Three_Years_On_Reformatted.pdf. Two-year retrospective review recommending principle amendments including explicit reproducibility requirement and clarification of domain-relevant standards.
- Iterative, Inc. DVC: Data version control. <https://dvc.org>, 2024. Open-source tool for ML experiment management, data versioning, and pipeline orchestration.

- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Daniel S. Katz, Neil P. Chue Hong, Tim Clark, Mercè Crosas, Bohdan B. Khomtchouk, John E. Kratz, David Leinweber, Kyle Niemeyer, Arfon Smith, and Patrick Vandewalle. Recognizing the value of software: A software citation guide. *F1000Research*, 9:1257, 2021. doi: 10.12688/f1000research.26932.2.
- Donald E. Knuth. Literate programming. *The Computer Journal*, 27(2):97–111, 1984. doi: 10.1093/comjnl/27.2.97.
- Johannes Köster and Sven Rahmann. Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19):2520–2522, 2012. doi: 10.1093/bioinformatics/bts480.
- Anna-Lena Lamprecht, Leyla Garcia, Mateusz Kuzak, Carlos Martinez, Ricardo Arcila, Eva Martin Del Pico, Victoria Dominguez Del Angel, Stephanie van de Sandt, Jon Ison, Paula Andrea Martinez, Peter McQuilton, Alfonso Valencia, Jennifer Harrow, Fotis Psomopoulos, Josep Ll. Gelpi, Neil Chue Hong, Carole Goble, and Salvador Capella-Gutierrez. Towards FAIR principles for research software. *Data Science*, 3(1):37–59, 2020. doi: 10.3233/DS-190026.
- Sam Lau and Philip J. Guo. The design space of LLM-based AI coding assistants: An analysis of 90 systems. In *IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, 2025. URL https://pg.ucsd.edu/publications/ai-coding-assistants-design-space_VLHCC-2025.pdf.
- Chris Lu, Jeff Clune, Jakob Foerster, Brian Lim, Robert Tjarko Lange, Yutian Tang, Shengran Mao, et al. The AI scientist: Towards fully automated open-ended scientific discovery. arXiv preprint arXiv:2408.06292, 2024. URL <https://arxiv.org/abs/2408.06292>.
- Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008. ISBN 978-0132350884.
- Gerard Meszaros. *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley, 2007. ISBN 978-0131495050.
- Laurenz Mädje and Martin Haug. Typst: A new markup-based typesetting system, 2023. URL <https://typst.app>. Programmable typesetting system with incremental compilation; open-sourced 2023.
- Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. doi: 10.1073/pnas.1708274114.
- Daniel Nüst, Markus Konkol, Edzer Pebesma, Christian Kray, Marc Schutzeichel, Holger Przibytzin, and Jörg Lorenz. Opening the publication process with executable research compendia. *D-Lib Magazine*, 23(1/2), 2017. doi: 10.1045/january2017-nuest.
- Open Source Security Foundation. SLSA: Supply-chain levels for software artifacts, v1.0. <https://slsa.dev/spec/v1.0/>, 2023. Accessed: 2026-03-19.
- OpenAI. Prism: AI-assisted research writing environment. <https://openai.com/prism>, 2026. Launched 2026-01-27 on GPT-5.2; provides context-aware manuscript assistance across notes, drafts, equations, and citations.
- Overleaf (Digital Science). Overleaf: Collaborative cloud L^AT_EX editor. <https://www.overleaf.com>, 2025. Real-time collaborative LaTeX editor used by 15M+ researchers across 4,000+ institutions.
- Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi: 10.1126/science.1213847.
- Research Software Alliance. Developing actionable guidelines for FAIR research software. ReSA Task Force, 2024. URL <https://zenodo.org/records/18877929>. Task Force launched December 2024 analyzing 17 FAIR4RS principles into 6 actionable categories; builds on FAIR-BioRS methodology.

- Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLOS Computational Biology*, 9(10):e1003285, 2013. doi: 10.1371/journal.pcbi.1003285.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. arXiv preprint arXiv:2302.04761, 2023. URL <https://arxiv.org/abs/2302.04761>.
- Joseph P. Simmons, Leif D. Nelson, and Uri Simonsohn. False-positive psychology: Undisclosed flexibility in data collection and analysis allows presenting anything as significant. *Psychological Science*, 22(11):1359–1366, 2011. doi: 10.1177/0956797611417632.
- Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*, pages 1393–1410, 2019. URL <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv preprint arXiv:2305.16291, 2023. URL <https://arxiv.org/abs/2305.16291>.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024a. doi: 10.1007/s11704-024-40231-1.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yunzhu Song, Bowen Li, Jasmine Li, Tao Zhang, Chuyi Ma, Ruohan Yu, Wangchunshu Yin, Karen Livescu, and Graham Neubig. OpenDevin: An open platform for AI software developers as generalist agents. arXiv preprint arXiv:2407.16741; published as OpenHands at ICLR 2025, 2024b. URL <https://arxiv.org/abs/2407.16741>.
- Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, Niklas Blomberg, Jan-Willem Boiten, Luiz Bonino da Silva Santos, Philip E. Bourne, et al. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016. doi: 10.1038/sdata.2016.18.
- Greg Wilson, Jennifer Bryan, Karen Cranston, Justin Kitzes, Lex Nederbragt, and Tracy K. Teal. Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6):e1005510, 2017. doi: 10.1371/journal.pcbi.1005510.
- Yihui Xie, J. J. Allaire, and Garrett Golemund. *R Markdown: The Definitive Guide*. Chapman and Hall/CRC, 2018. ISBN 978-1138359338. doi: 10.1201/9781138359444.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*, 2023. URL <https://arxiv.org/abs/2210.03629>.

END OF TRANSMISSION

Release: v1.0.9 · DOI 10.5281/zenodo.20419007 · SHA-256 535bd80943d0... · pairing complete

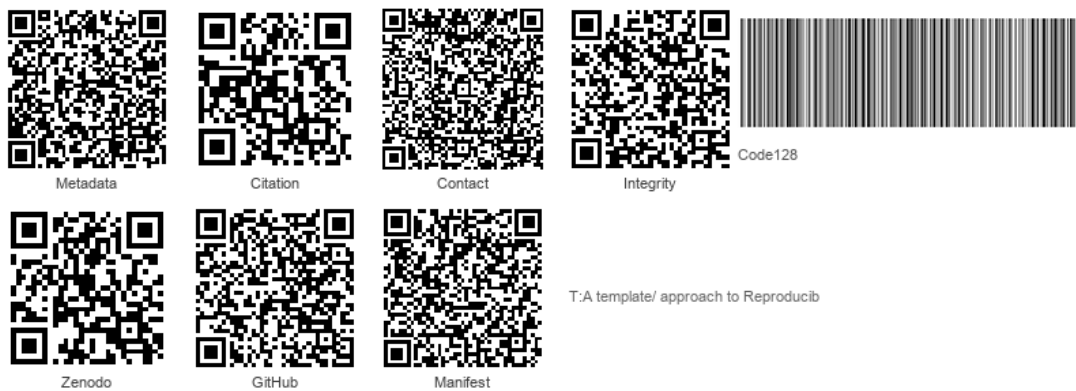


Figure 3: Integrity QR strip

Prior: v1.0.7 · 10.5281/zenodo.20419007 · cc674248... · v1.0.8 · 10.5281/zenodo.20932076 · b9bc5cf3...